

FORGE

Turning a painful workflow into a shipped internal tool — with almost no one in the loop but the people who should be

Design notes — AI Implementation Intern

What Forge Does

Good ideas for internal tools come up constantly, in a ticket, on a call, in a passing Slack message, and most of them just die there because turning a complaint into a working tool takes real effort that nobody has spare time for. Forge is my answer to that gap: it watches for these signals on its own, decides which ones are worth acting on, writes the spec, designs the tool, and builds it, checking in with a person only at the three moments where a wrong call would actually cost something.

Four stages: Find, Define, Design, Build. Three checkpoints, one after each of the first three. Everything else runs on its own.

How Forge Finds Problems

Where the signal comes from, and why those sources

Three sources feed it: Jira tickets, transcripts from recurring team calls, and a Slack channel people can post directly into. I picked these because they're not redundant. Jira gives frequency for free, if the same kind of ticket shows up three times, that's already a signal. Transcripts catch the stuff nobody bothers filing a ticket for, someone just says out loud that a task eats half their Friday. And the Slack channel is the easiest signal to trust, because someone chose to write it down.

I left out full Slack history, email, and code repos. More sources sounds better, but it's mostly noise for the extra signal you get, and email raises privacy questions I didn't want to open. Precision over recall, on purpose, and it means Forge will miss real problems that never surface in these three places. Accepted trade-off, not an oversight.

What makes something worth pursuing, and how ties get broken

Nothing gets acted on from a single mention. Every incoming item gets classified into a small structured stub by a cheap model, and once a week a batch job clusters stubs by similarity, so the same complaint showing up in a ticket, a call, and a Slack post collapses into one corroborated cluster instead of three easy-to-dismiss data points. Each cluster is scored on frequency, people affected, estimated hours saved, and whether the platform could plausibly act on it. Clear the bar, you become a candidate for review. Miss it, you're not thrown away, you sit and get re-scored next week. When more than one candidate clears the bar in the same cycle, that same score is what ranks them, so the reviewer sees a reasoned order, not just a pile of options.

Concretely, each surviving event gets classified into a stub like this:

```
{
  "problem": "manual invoice/PO reconciliation",
  "team": "finance",
  "source_type": "ticket",
  "confidence": 0.82,
  "external_id": "JIRA-4521"
}
```

Stubs get embedded (a small embedding model is enough here, this isn't a semantic-search problem) and clustered weekly using cosine similarity, threshold around 0.80-0.85 in practice, tuned to catch reworded duplicates without merging unrelated complaints. Score is a plain weighted sum, not a model call:

```
score = 0.35*frequency_recency
       + 0.25*people_affected
```

```
+ 0.25*est_hours_saved
+ 0.15*feasibility
```

Weights above are a starting point, not something I'd defend as tuned, real usage data should set these. The point is that the formula is inspectable and adjustable, not a black box the reviewer has to trust blindly.

How it actually runs

Collection is continuous. Scoring runs weekly. If it scored in real time, the reviewer would either get flooded or start ignoring the alerts, either way the checkpoint stops meaning anything. The one exception is the suggest-channel: a direct post already carries intent, so it can feed into the very next scoring cycle instead of waiting around, but it still has to clear the same bar as everything else.

From Problem to Shipped Tool

Writing the spec

Once a candidate is approved, all Forge has is a paragraph and a pile of evidence, not enough to write a real spec from. So it doesn't try to in one shot. It first goes back for the full raw material, the actual ticket comments, the full transcript, the whole thread. Separately, it pulls internal context, existing docs, tools the team already uses, known constraints, kept apart from the evidence on purpose, because evidence tells you what hurts today and context tells you what a solution has to fit inside.

If something's still missing that would genuinely change scope, it asks a person up to three targeted questions, capped, so it never turns into an interview. Then it drafts, every requirement cites the evidence behind it, anything it can't ground gets written down as a flagged assumption instead of invented. Before a human sees any of it, a self-review pass checks the scope is real, the requirements are testable, and nothing snuck in that doesn't actually serve the original problem. Then it builds a mockup and runs one more check comparing the spec against the mockup directly, since two documents can each look fine alone and still contradict each other.

Every requirement in the draft looks like this, not just prose:

```
{
  "id": "R4",
  "text": "System auto-assigns unclaimed leads after 4h",
  "evidence": ["JIRA-4521", "TRANSCRIPT-18#12:04"],
  "status": "grounded"
}

{
  "id": "A2",
  "assumption": "Only managers approve requests",
  "confidence": "low",
  "reason": "no approval workflow doc found"
}
```

That schema is what makes self-review mechanical rather than vibes-based: it can check every R-id has a non-empty evidence array before a human ever sees the draft, and flag anything that doesn't.

Building it

The build stage breaks the spec into tasks, each one tied back to the requirement it implements, and works out which tasks depend on which others before writing a line. That dependency map is what makes the failure handling actually sensible: if a task other tasks depend on can't be fixed after a few retries, the whole build stops right there, because continuing means building on something broken. If nothing depends on it, it gets logged as a known gap and the build keeps going. A running health score also catches the case where several unrelated things fail, since that usually means something's off with the spec itself, not bad luck three times over.

Each planned task carries its DAG position explicitly:

```
{
  "task_id": "B7",
  "requirement": "R4",
  "depends_on": ["B2"],
```

```

"depth": 1,
"criticality": 3,
"parallel_group": "B"
}

```

criticality is just the count of tasks that transitively depend on this one, it's what decides halt-vs-continue on an unresolved failure. Health score after a failure is a simple decay, weighted by that same criticality plus retry count and verification confidence:

```

health -= (5 + 10*criticality) * (1 - verification_confidence)
if health < 60: halt_build()

```

Same caveat as the discovery weights: illustrative, not tuned. What I'd actually defend is the mechanism, dependency-aware halting beats a flat retry count, not these specific numbers.

By the time a build reaches a person, it's already been checked against the full spec and mockup, with a report on exactly what did and didn't make it in, and why.

How Forge catches its own bad work

Two separate checks, for two separate failure modes. Self-review catches a spec that's individually weak, vague scope, unmeasurable success criteria, unsupported claims, and sends it back to drafting automatically before a person ever sees it. The consistency check catches something else: a spec and mockup that are each fine alone but contradict each other, which self-review would never notice since it only ever reads one document at a time. In Build, per-task verification catches a single bad task early, and the conformance check catches the assembled app not matching the whole spec, same two-layer idea, applied further downstream.

What happens when it genuinely doesn't know

Two different moments get handled differently. While gathering evidence and context, if something's missing that would actually change scope, it asks a person up to three questions instead of guessing, capped so it can't turn into an interrogation. While drafting, if a detail isn't backed by anything gathered and wasn't resolved by a question, it doesn't invent it, it writes it down as a flagged assumption and lets the person catch it at review instead of finding a made-up number buried in the document later.

Where People Stay in Control

Where the checkpoints sit, and why each one specifically

Three checkpoints, and they're not the same review three times. The first sits after Find, because Forge can't know things like a team being mid-reorg, that's context that just doesn't live in a ticket, and a wrong call here costs almost nothing since no drafting effort has been spent yet. The second sits after Define, because a spec can be fully grounded and still be the wrong thing to build, that's a product-fit judgment, not a process check, and the system already verified the process parts itself. The third sits before anything goes live, because even a clean build can have known gaps, and deciding whether those gaps are acceptable is a risk call a person should own, not the system.

What the person is actually asked to do at each one

The first is quick triage, a paragraph, the evidence, a score, approve or reject in under a minute, closer to skimming an inbox than reading a document. The second is a real design review, the full spec and mockup together, since a spec is words describing behavior and a mockup shows it, and reviewing them side by side beats imagining one from the other. The third is closer to acceptance testing, a live, working preview alongside a report of what did and didn't make it in, closer to how someone signs off on a finished feature than how they'd review a proposal.

How it waits, and what it does with the answer

Between checkpoints nothing runs ahead of an unmade decision, no polling, no timeout, it just parks and resumes exactly where it left off once a person responds, whether that's a dashboard click or a Slack button. A rejection doesn't restart everything, it goes back to whichever stage could actually be at fault, a rejection at the third checkpoint returns to build planning, not all the way back to the spec, since the spec and mockup were already signed off and whatever went wrong was in execution.

Keeping people from becoming the bottleneck

Two things do the work here. Each checkpoint is only as heavy as it needs to be, the first one is fast on purpose since it's the one that fires most often. And everything that can plausibly be resolved without a person already has been by the time a checkpoint is reached, self-review, the consistency check, automated recovery, all exist so the person is only ever making a real judgment call. The honest risk: this only works if those earlier checks are actually reliable. If they're not, a person ends up doing that work anyway, just later, disguised as review.

If I had to actually build it

The brief treats the platform as a black box on purpose, so none of this is load-bearing, but it's the answer if someone asks how I'd build it for real: Temporal for orchestration, since the whole pipeline depends on durable wait-for-a-human states plus bounded retries, which is exactly what it's built for. LangGraph for each agent's internal steps. Postgres plus pgvector for state and clustering, so it's one database instead of three. Claude with routing by stage, cheap model for high-volume classification, stronger model for drafting and review. Structured output enforced through tool-use schemas, not hopeful prompting, which is what actually makes the citation and DAG examples above reliable rather than aspirational. A thin Next.js dashboard plus a Slack bot for approve or reject at each checkpoint.

Trade-offs, Risks, and What I'd Do Next

Stronger reasoning costs more and is slower, so the expensive model calls are reserved for the low-volume steps that need real judgment, spec drafting, review, build diagnosis, while the high-volume step of classifying every incoming ticket runs on something cheap.

The main ways this could fail, and what actually catches it

False positives at the discovery stage, handled by requiring corroboration across sources rather than trusting one mention. A spec quietly inventing details, handled by forcing every claim to cite real evidence. A build silently compounding a broken assumption, handled by the dependency-aware halt logic. Alert fatigue at the checkpoints, handled by only ever surfacing a small, weekly, already-corroborated batch instead of firing on every promising item. None of these are fully solved, they're reduced, which is why the checkpoints still exist rather than being decoration.

Privacy and access

Since Forge reads real internal tickets, transcripts, and docs, it has to inherit whatever access controls already exist, pulling data under the requesting user's own permissions rather than a blanket credential, keeping anything sensitive inside the scope it was already authorized for, and logging every retrieval so it's reconstructable later.

What I'm assuming

The internal platform can actually run multi-step workflows, call tools, hold state, and pause and resume on human input, the way the brief describes. If any of that is weaker in practice, a lot of this design stops working. Beyond that: Jira, Slack, and transcription integrations stay reliably available with proper scoped access, internal docs exist and are current enough for context collection to be worth doing, and there's an actual assigned person behind each checkpoint, not just whoever happens to see the Slack ping.

What's deliberately out of scope: Forge never takes action on its own, no auto-reassigning tickets, no auto-sending reminders, since that would undercut the entire point of the checkpoints. A full map of how work flows across every internal tool was considered and cut, it's closer to its own product than something a focused build should try to own.

How I'd know it's working

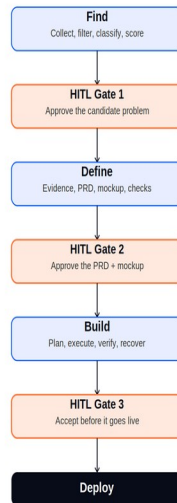
Approval rate at each checkpoint, tracked separately, tells you different things: a low Gate 1 approval rate means scoring is too loose, a high Gate 2 rejection rate specifically means Define is solving the wrong version of the problem. Time from a problem first showing up in the evidence to a working build reaching Gate 3 is the real measure of whether Forge is closing the gap it's meant to close. Recovery's self-resolution rate tells you whether the human is actually staying out of the way for routine failures. And how often a flagged assumption in a PRD turns out to be wrong at review is a decent check on whether the grounding discipline is real or just theater.

What I'd do with more time: check the clustering logic against real historical data instead of made-up examples, tune the build health scoring against actual failure data instead of illustrative numbers, and eventually feed checkpoint decisions back into the scoring so Forge's sense of what's worth pursuing gets better with use, carefully, so it doesn't just calcify one reviewer's blind spots.

How It Fits Together

The pipeline

The full sequence end to end — three autonomous stages, three checkpoints in between.



The components

Where the diagram above shows order, this shows what's actually talking to what. Everything flows through one shared state store and one orchestrator, all three agent pools call into the same model service and platform connector, and the human interface is one shared surface used at all three checkpoints, not three separate ad hoc reviews.

